BRANDROOTS - ROUND 2 BUILD REPORT

Inkloom AI Hackathon Round 2 | Team: **vettriselvam190** | Status: **Shipped & Verified Application**

Executive Summary: *BrandRoots is an intelligent pre-publication brand-governance platform. Unlike conventional linters that merely flag isolated mistakes in individual drafts, BrandRoots introduces the* ***Guideline Debugger****: an engine that investigates whether repeated content mistakes are symptoms of vague, contradictory, or outdated brand instructions. In Round 2, the team moved beyond the proposal to deliver a working full-stack implementation with React 19, FastAPI, hybrid compliance checks, safe PyMuPDF/python-docx extraction, persistent rule versioning, and an immutable governance audit trail.*

1. The Problem and Target Users

Modern organizations publish high-velocity content through dispersed employees, external agencies, freelancers, and generative AI tools. This causes messaging drift: outdated pricing tiers, unauthorized medical claims, deprecated terminology, and missing regulatory disclosures. When teams repeatedly make the same mistake, reviewers continually spend hours correcting symptoms rather than the root cause: an ambiguous or boundary-lacking guideline.

Target Users:

• **Content Creators:** Need rapid, explainable feedback before submission, with 1-click suggested corrections.
• **Compliance & Legal Reviewers:** Need traceable citations to approved policies with strict approval gates.
• **Brand Governance Owners:** Need a system that learns from review history, surfaces recurring rule ambiguities, and manages versioned guidelines.

2. Actual Product Architecture

BrandRoots is architected as a modular, three-tier cloud-ready system:

• **Frontend Layer:** Single Page Application built on React 19, TypeScript, and Tailwind CSS. Features role switching (Creator, Reviewer, Brand Owner), live canvas editing, split review workbench, and the signature Guideline Debugger interface.
• **Backend Layer:** High-performance Python 3.11 service using FastAPI and Pydantic v2. Provides REST endpoints for document ingestion, review execution, status transitions, and clustering.
• **Ingestion & Security Engine:** PyMuPDF (fitz) for PDF extraction and python-docx for DOCX files, equipped with size limits (10MB) and a prompt-injection regex guard that detects instruction-override attempts.
• **Data & Provenance Layer:** Relational database (SQLite local development default, PostgreSQL / Supabase cloud-ready) tracking 11 core tables with immutable version snapshots and append-only audit events.

3. Features Implemented and Verified

Feature Module	Implementation Details	Verification Status
Role-Based Access Control	Creator, Reviewer, and Brand Owner roles enforced on backend. Policy prevents creators from approving their own work.	VERIFIED (pytest + UI)
Hybrid Compliance Engine	Combines deterministic matching (exact terms, pricing conflicts, required disclaimers) with contextual heuristic checks (guaranteed outcomes).	VERIFIED (100% test pass)
Structured Explainability	Every finding includes severity, plain-language explanation, source document citation, rule version, and suggested correction.	VERIFIED (Schema validated)
Interactive Correction	Creators can accept suggestions with 1-click in-place draft patching, triggering a new document version snapshot.	VERIFIED (E2E & UI)
Content Lifecycle	Full state machine: Draft -> Submitted -> In Review -> Changes Requested -> Resubmitted -> Approved / Rejected.	VERIFIED (API & transitions)
Guideline Debugger	Clusters recurring violations across drafts, isolates ambiguous rules, proposes draft clarifications, and supports Brand Owner approval.	VERIFIED (Signature flow)
Document Ingestion	Safe extraction of PDF and DOCX files with prompt injection detection and file size validation.	VERIFIED (PyMuPDF + docx)
Immutable Audit Trail	Append-only log recording all rule changes, approvals, comments, and finding resolutions.	VERIFIED (DB & UI feed)

4. The Signature Feature: Guideline Debugger

The **Guideline Debugger** embodies BrandRoots' core thesis: *recurring mistakes are data*. Instead of treating repeated violations as creator incompetence, the debugger monitors review history to locate systemic ambiguities in the company's guidelines.

The Debugger Workflow:
1. **Aggregation:** The engine clusters stored review findings by rule ID across all documents.
2. **Evidence Extraction:** When recurring violations are spotted (e.g., 5 drafts promising 'guaranteed results'), the debugger extracts exact quotes and calculates frequency.
3. **Root Cause Hypothesis:** It analyzes why the guideline failed (e.g., RULE-CLAIM-01 encouraged bold persuasion but lacked an explicit boundary regarding outcome guarantees).
4. **Draft Proposal:** It generates a proposed clarification (e.g., RULE-CLAIM-01 v1 -> v2) adding an explicit boundary clause.
5. **Human-in-the-Loop Approval:** An authorized Brand Owner explicitly reviews, edits, and approves the proposal. The active guideline version is incremented to v2, ensuring future reviews enforce the clarified rule while historical review snapshots remain intact.

5. Technology and Implementation Details

• **Backend Engine:** Built with FastAPI 0.115, Python 3.11, Pydantic 2.13, and SQLAlchemy 2.0. Clean architecture separating schemas, ORM models, routers, and domain services.
• **Compliance Engine (LLM & Heuristic):** Provider interface supports external LLM providers (OpenAI/Anthropic/Gemini) with structured outputs, paired with an offline, transparent local_hybrid_deterministic engine that delivers 100% reproducible checks without external key dependencies.
• **Frontend Architecture:** React 19 SPA bundled with Vite 6. Features deep navy surfaces (#0A1424), restrained teal accents (#0D9488), accessible contrast, responsive layout, and role-based views.
• **Security & Ingestion:** Byte-level file inspection via PyMuPDF and python-docx. Prompt injection guard neutralizes instruction hijacking markers.

6. What Changed from Round 1 and Why

In the Round 1 proposal, BrandRoots was conceptualized as a multi-format governance platform. For Round 2, the team made deliberate, principled adjustments:
1. **Focused Scope on High-Value Text and Documents:** Following the build plan, text editor and PDF/DOCX review workflows were prioritized over multimodal image analysis to ensure the core Guideline Debugger and approval lifecycle were fully operational and rigorously tested.
2. **Decoupled Local Engine:** To guarantee complete testability without external API fragility, an intelligent local hybrid engine was implemented. External integrations remain configurable via environment variables without faking connectivity.
3. **Relational Data Integrity:** Rather than flat JSON blobs, an 11-table relational schema was implemented with foreign key constraints, versioned snapshots, and an append-only audit trail.

7. Test Results and Measurable Findings

Test Suite	Scope & Criteria	Executed Tests	Pass Rate
Backend Unit & Integration	Auth RBAC, Guideline CRUD, Pricing checks, Lifecycle transitions, Version bumps, PyMuPDF/docx extraction, Debugger clustering	19 pytest tests	**100% (19/19 PASSED)**
Frontend Component Tests	Badge rendering, FindingCard action dispatch, severity mapping	2 vitest tests	**100% (2/2 PASSED)**
End-to-End Demo Workflow	Complete 9-step simulation: creation -> detection -> correction -> review -> clustering -> ambiguity confirmation -> v2 approval -> audit	1 automated script (verify_e2e_demo.py)	**100% (9/9 Steps PASSED)**
Frontend Production Build	Vite production bundling and TypeScript static verification	npm run build	**0 errors (Built in 14.8s)**

8. What Does Not Work Yet (Known Limitations)

In adherence to hackathon integrity guidelines, current limitations are transparently documented:
• **Multimodal Image Review:** BrandRoots currently inspects textual content, PDFs, and DOCX files. Image logo positioning, color contrast in uploaded graphics, and visual alt-text audits are not yet supported.
• **External Cloud Auth / Public Hosting:** The current build runs on a fully functional local JWT session engine with seeded demo personas. Live deployment to public Vercel/Render instances requires cloud credentials and is ready for deployment following docs/DEPLOYMENT.md.
• **Automated Embedding Clustering:** Finding clustering currently groups by rule ID, category, and lexical patterns; semantic vector clustering using pgvector is reserved for Round 3.

9. Verified Repository and Application URLs

• **Local Application:** Frontend live at http://localhost:5173 | Backend live at http://localhost:8000
• **Source Code Workspace:** Clean repository prepared in scratch/brandroots with complete source code, tests, and documentation.
• **Deployment Status:** Pre-configured for Render (backend) and Vercel (frontend) via Supabase PostgreSQL. Deployment instructions documented in docs/DEPLOYMENT.md.

10. Commitments for Round 3

The team commits to three substantial improvements for Round 3:
• **Improvement 1 — Multimodal brand review:** Add reliable visual asset checks for uploaded campaign images, including logo placement, brand color consistency, and accessibility checks such as automated alt-text guidance.
• **Improvement 2 — Stronger evidence and claim verification:** Build a version-aware evidence retrieval and claim-verification pipeline that distinguishes verified, contradicted, expired, and unsupported claims, with source-level traceability.
• **Improvement 3 — Measurable governance intelligence:** Improve the Guideline Debugger with semantic clustering, reviewer feedback loops, ambiguity confirmation metrics, and empirical evaluation against a labelled test dataset.